Compiler-Planned Knowledge Distillation (CPKD)

Fused Teacher-Student Training Through Shared Kernel Synthesis, Compile-Time Scheduling, and Source-AD-Generated Distillation Losses in NeuralScript

Authors: Brandon Wiemer
Date: April 2026
Status: Research Proposal

Abstract

We propose **CPKD (Compiler-Planned Knowledge Distillation)**, a compiler-native framework where knowledge distillation — training a smaller student model to mimic a larger teacher — is treated as a **joint compilation problem** rather than two independent model executions. In PyTorch, distillation requires loading both teacher and student models into GPU memory, running separate forward passes, computing a combined loss (hard label CE + soft label KL divergence), and managing two sets of activations. CPKD exploits NSL's whole-program compilation to fuse the teacher and student forward passes into shared kernels, eliminate the full teacher logit tensor from HBM, overlap teacher inference with student backward via compile-time scheduling, and auto-generate optimal distillation losses via source AD. The result: distillation that approaches the memory cost of training the student alone, with the quality benefits of a teacher signal.

---

1. The Memory Crisis in Knowledge Distillation

1.1 Standard Distillation Memory

For white-box distillation of a 7B teacher into a 1B student (LLaMA-class):

```
Teacher model (frozen, BF16):           14.0 GB
Student model (trainable, BF16):         2.0 GB
Student FP32 master weights:             4.0 GB
Student optimizer (AdamW FP32):          8.0 GB
Teacher activations (for logits):        8.0 GB  ← saved for KL loss backward
Student activations:                     3.0 GB
Teacher logits [batch, seq, vocab]:      batch × 2048 × 32000 × 4 = 256 MB per
sample
Student logits:                          256 MB per sample
Gradient buffer:                         2.0 GB
──────────────────────────────────────────────
TOTAL:                                  ~41+ GB
```

The dominant cost is the teacher. The teacher model (14 GB) plus its activations (8 GB) consume 22 GB — more than the entire student training footprint. And this teacher is frozen; we never compute gradients for it.

1.2 Current Solutions (and their limitations)

- **Offline distillation:** Pre-compute teacher logits and save to disk. Eliminates teacher from GPU but requires massive storage (256 MB × dataset_size) and loses the ability to do on-policy distillation.
- **DeepSpeed ZeRO:** Shard teacher across GPUs. Helps with multi-GPU but doesn't reduce single-GPU cost.
- **Top-k logit truncation:** Store only top-k teacher logits instead of full vocab. Loses information from the tail distribution.

None of these address the fundamental problem: the teacher and student are treated as two separate computational graphs with no shared analysis.

---

2. CPKD: Five Compiler-Native Innovations

2.1 Innovation 1: Fused Teacher-Student Forward Pass

When teacher and student share architectural components (e.g., both are transformer decoders), the compiler identifies **shared computation** and generates fused kernels.

Shared embedding: If teacher and student use the same tokenizer and embedding dimension (or a projection), the embedding lookup is computed once:

```
nsl

distill(teacher=t, student=s):
    # Compiler detects: t.embed.vocab_size == s.embed.vocab_size
    # Generates: one embedding lookup, projects to both d_model dimensions
    step(batch):
        let shared_embed = embed_lookup(batch.input_ids)  # one HBM read
        let t_input = project(shared_embed, t.d_model)    # if dimensions differ
        let s_input = project(shared_embed, s.d_model)
```

Shared attention computation: For layers where teacher and student have compatible dimensions, the Q/K/V projections can share the input activation read:

```
Standard:
  Teacher layer l: read x_t from HBM → Wq_t @ x_t → ...
  Student layer l: read x_s from HBM → Wq_s @ x_s → ...
  = 2 HBM reads of x

CPKD fused:
  Read x_t from HBM (once)
  Wq_t @ x_t → teacher Q (in SMEM)
  If student layer l maps to teacher layer l:
    Read x_s from HBM (once)
    Wq_s @ x_s → student Q (in SMEM)
    Both Qs proceed through attention in the same kernel launch
```

For a 7B→1B distillation where the student has 16 layers mapping to every other teacher layer, this saves 16 kernel launches and 16 HBM reads.

2.2 Innovation 2: Fused KL-CE Loss (Never Materialize Teacher Logits)

The problem: The standard distillation loss is:

```
L = α × CE(student_logits, labels) + (1-α) × T² × KL(softmax(teacher_logits/T),
softmax(student_logits/T))
```

This requires materializing BOTH teacher and student logits as [batch, seq, vocab] tensors in HBM. For vocab=32000, seq=2048, batch=4: **2 × 4 × 2048 × 32000 × 4 bytes = 2 GB** just for the logit tensors.

CPKD's approach: Extend NSL's existing fused linear cross-entropy (from CSHA research) to include the KL divergence term. The fused kernel computes the LM head matmul for BOTH teacher and student, and immediately computes the combined loss — without ever writing the full logit tensors to HBM:

```
Fused KL-CE Distillation Kernel:
  Inputs:
    student_hidden [B, S, d_student]
    teacher_hidden [B, S, d_teacher]
    student_lm_head [d_student, vocab]
    teacher_lm_head [d_teacher, vocab]
    labels [B, S]
    alpha, temperature

  Output: scalar loss, student gradient w.r.t. hidden

  for vocab_tile in 0..ceil(vocab / TILE):
    # Teacher logits tile (no grad needed)
    t_logits_tile = teacher_hidden @ teacher_lm_head_tile  # [B, S, TILE] in SMEM
    t_probs_tile = softmax(t_logits_tile / T)               # in SMEM

    # Student logits tile (grad needed)
    s_logits_tile = student_hidden @ student_lm_head_tile   # [B, S, TILE] in SMEM
    s_log_probs_tile = log_softmax(s_logits_tile / T)       # in SMEM

    # Accumulate KL divergence (tiled online computation)
    kl_loss += sum(t_probs_tile * (log(t_probs_tile) - s_log_probs_tile))

    # Accumulate CE loss for label positions
    for (b, s) in batch_seq_positions:
      if labels[b, s] is in this vocab_tile:
        ce_loss += -log_softmax(s_logits_tile[b, s, labels[b,s] - tile_offset])

    # Accumulate gradient of combined loss w.r.t. student hidden
    # (backprop through student_lm_head_tile matmul)
    d_loss_d_s_logits = alpha * d_CE + (1-alpha) * T * (s_probs - t_probs)
    grad_hidden += d_loss_d_s_logits @ student_lm_head_tile.T

  # Final output
  loss = alpha * ce_loss + (1-alpha) * T² * kl_loss
  # grad_hidden written to HBM (same size as student_hidden — not vocab-sized)
```

Memory savings:

- Teacher logits tensor: **eliminated** (never in HBM)
- Student logits tensor: **eliminated** (never in HBM)
- Saved: 2 × batch × seq × vocab × 4 bytes = **2 GB at batch=4, seq=2048, vocab=32000**

Backward integration: The kernel computes `grad_hidden` for the student inline. Source AD (M40) generates the rest of the student backward automatically. The teacher has no backward — its weights are frozen.

2.3 Innovation 3: Compile-Time Teacher Scheduling

The problem: In standard distillation, the training loop is:

```
teacher_logits = teacher.forward(x)    # GPU busy with teacher
student_logits = student.forward(x)    # GPU busy with student
loss = kl_ce_loss(teacher_logits, student_logits, labels)
loss.backward()                        # GPU busy with student backward
optimizer.step()
```

The teacher forward, student forward, and student backward are all sequential. The teacher forward is pure inference (no gradient) but still occupies the GPU during student idle time.

CPKD's approach: The compiler schedules teacher and student computation to **overlap**. Using WGGO's communication scheduler (from CPDT), adapted for compute-compute overlap instead of compute-communication overlap:

```
CPKD Scheduled Pipeline (per micro-batch):

  Layer 0:
    [Stream 0] teacher.layer_0.forward(x)  → t_hidden_0
    [Stream 1] student.layer_0.forward(x)  → s_hidden_0
    (overlap: teacher and student compute simultaneously)

  Layer 1:
    [Stream 0] teacher.layer_1.forward(t_hidden_0) → t_hidden_1
    [Stream 1] student.layer_1.forward(s_hidden_0) → s_hidden_1

  ...

  After all layers:
    [Stream 0] fused_kl_ce(t_hidden_final, s_hidden_final) → loss + grad
    [Stream 1] student.backward() begins (FASE-fused, per-layer)
```

For the 7B→1B case: the teacher has 32 layers, the student has 16. The compiler pairs teacher layers 0+1 with student layer 0, teacher layers 2+3 with student layer 1, etc. Two CUDA streams run concurrently — the GPU's SMs split between teacher and student.

When this helps: When the teacher is compute-bound and the student is memory-bound (or vice versa), the overlapped execution fills idle resources. The cost model determines whether overlap improves total time; if not (both compute-bound), the compiler falls back to sequential.

2.4 Innovation 4: Teacher Logit Compression via Spectral Analysis

The problem: Top-k logit truncation is a common approximation (store only the top-k teacher logits, discard the rest). But choosing k is ad-hoc — too small loses information, too large wastes memory.

CPKD's approach: When teacher weights are available, the compiler analyzes the teacher's LM head weight matrix spectral profile to determine the **effective vocabulary rank** — how many logits carry meaningful information:

```
teacher_lm_head: [d_model, vocab]
singular_values = svd(teacher_lm_head).S
energy_90 = find k such that sum(S[:k]²) / sum(S[:]²) ≥ 0.90

# If energy_90 = 800, then the teacher's output distribution has
# ~800 effective dimensions — top-k=800 captures 90% of the signal
```

The compiler sets k per the spectral analysis and generates the fused KL-CE kernel with this k as a compile-time constant. The top-k selection happens on-chip during the tiled matmul (as in the fused decode-sample kernel from CFIE) — no full logit tensor needed.

Typical values: For LLaMA-7B, the effective rank is ~500-2000, meaning k=1000 captures the vast majority of the teacher signal. This is 32× smaller than the full vocab=32000.

2.5 Innovation 5: CEP-Guided Student Architecture Design

The problem: Choosing the student architecture is manual guesswork. Users typically halve the teacher dimensions (d_model/2, n_layers/2, n_heads/2) with no principled basis.

CPKD's approach: CEP (from Paper 5) designs the student architecture by searching for the optimal sub-architecture within a target parameter budget:

```
nsl

@cpkd_design_student(
    teacher = TeacherModel,
    target_params = 1B,
    target_hardware = rtx5070ti,
    constraints = { latency_per_token < 5ms }
)
model Student:
    # CEP search determines: d_model, n_layers, n_heads, n_kv_heads, d_ff
    # Compilation oracle evaluates each candidate for hardware efficiency
    # WGGO ILP selects the architecture that maximizes:
    #   representational capacity × hardware utilization / parameter count
```

CEP evaluates hundreds of candidate student architectures in seconds using the compilation oracle, selecting the one that maximizes quality-per-parameter on the target hardware.

Layer mapping: The compiler also determines which teacher layers map to which student layers for intermediate feature matching. Using importance scores from the teacher's weight analysis, it selects the k most important teacher layers (where k = student layers) and generates feature projection losses automatically via source AD.

---

3. Source-AD-Generated Distillation Losses

NSL's source AD (M40) can automatically generate backward passes for any differentiable computation. CPKD exploits this for complex distillation losses that would require manual gradient derivation in PyTorch:

3.1 Layerwise Feature Distillation

```nsl
distill(teacher=t, student=s):
    step(batch):
        # Forward both models, capturing intermediate representations
        let t_hiddens = t.forward_with_intermediates(batch.input_ids)
        let s_hiddens = s.forward_with_intermediates(batch.input_ids)

        # Feature matching losses (auto-generated by source AD)
        let feature_loss = 0.0
        for (t_layer, s_layer) in layer_mapping:
            # Projection if dimensions differ
            let t_proj = project(t_hiddens[t_layer], s.d_model)
            let s_feat = s_hiddens[s_layer]
            feature_loss += cosine_distance(t_proj, s_feat)

        # Combined loss
        let output_loss = fused_kl_ce(t_hiddens[-1], s_hiddens[-1], batch.labels, al
        let total_loss = output_loss + 0.1 * feature_loss
```

Source AD generates the backward for `cosine_distance`, the projection layers, and the fused KL-CE — all composed into a single backward function. The dead gradient elimination from WRGA ensures that only the student parameters receive gradients; the teacher's backward paths are physically absent.

3.2 Attention Transfer

nsl

```
# The compiler can match teacher and student attention patterns
let attn_loss = 0.0
for (t_layer, s_layer) in layer_mapping:
    # Teacher attention weights [B, H_t, S, S]
    # Student attention weights [B, H_s, S, S]
    let t_attn = t.layers[t_layer].attn_weights  # captured during forward
    let s_attn = s.layers[s_layer].attn_weights

    # Average over heads if head counts differ
    let t_avg = mean(t_attn, dim=1)  # [B, S, S]
    let s_avg = mean(s_attn, dim=1)
    attn_loss += mse(t_avg, s_avg)
```

In standard CSHA-fused attention, attention weights are not materialized in HBM (they stay in SMEM). For distillation, the compiler generates a CSHA variant that **writes attention weights to a compact buffer** only for the mapped layers, not all layers. This is a compile-time decision based on the `layer_mapping` in the distill block.

---

4. WGGO Integration

CPKD adds distillation-specific decisions to the WGGO ILP:

```
New WGGO variables per layer:
  feature_match[l] ∈ {0, 1}    # enable feature matching at layer l?
  attn_transfer[l] ∈ {0, 1}   # enable attention transfer at layer l?
  teacher_stream[l] ∈ {0, 1}  # overlap teacher layer l with student?

New constraints:
  # Feature matching requires saving teacher hidden state (memory cost)
  Σ_l feature_match[l] × d_teacher × seq × 2 ≤ feature_memory_budget

  # Attention transfer requires saving attention weights (memory cost)
  Σ_l attn_transfer[l] × batch × n_heads × seq² × 4 ≤ attn_memory_budget

  # Teacher-student overlap feasibility (sufficient SMs for both)
  teacher_stream[l] = 1 → sufficient_sms_for_overlap(l)
```

---

5. NSL Language Surface

nsl

```
# Full distillation pipeline
let teacher = LLaMA7B.load("teacher.nslm", weights="teacher.safetensors", frozen=tru
let student = LLaMA1B.load("student.nslm")  # or CEP-designed

distill(teacher=teacher, student=student, epochs=3):
    optimizer: AdamW(lr=1e-4, weight_decay=0.01)
    loss:
        alpha = 0.5           # CE weight
        temperature = 2.0     # KL temperature
        feature_layers = auto # WGGO selects which layers to match
        feature_weight = 0.1
        attn_transfer = false # disable attention matching

    data:
        source = train_data
        batch_size = 4
        packing = true         # PCA applies to both teacher and student

    step(batch):
        # This entire block compiles to a fused pipeline:
        # teacher forward (overlapped) → student forward → fused KL-CE loss → studen
        let loss = distill_step(teacher, student, batch.input_ids, batch.labels)
```

bash

```
$ nsl build --distill --teacher-weights teacher.safetensors --target rtx4090 distill

=== CPKD Distillation Build Report ===
Teacher: LLaMA-7B (5.8B params, frozen)
Student: LLaMA-1B (1.1B params, trainable)
Target: RTX 4090 (24GB)

Optimizations:
  [1] Fused KL-CE: teacher+student logits never materialized (saves 2 GB)
  [2] Teacher logit compression: k=800 (spectral rank), 32× reduction
  [3] Teacher scheduling: overlapped on layers 0-15 (sufficient SMs)
  [4] Feature matching: layers 4, 12, 20, 28 (WGGO selected)
  [5] Dead gradient elimination: teacher backward absent from binary
  [6] FASE: student optimizer fused per-layer
  [7] PCA: packing applied to both teacher and student attention

Memory breakdown:
  Teacher weights (BF16, frozen):      11.6 GB
  Student weights (BF16):               2.2 GB
  Student FP32 master weights:          4.4 GB
  Student optimizer (CPDT mixed):       3.3 GB
  Teacher activations (4 feature layers): 0.5 GB
  Student activations (FASE + CSHA):    0.8 GB
  Gradient (FASE: 1 layer at a time):   0.1 GB
  Logit tensors:                        0 (fused)
  ──────────────────────────────────────────
  TOTAL:                               22.9 GB  ← fits on 24GB RTX 4090!

Standard PyTorch:                      41+ GB   ← requires 48GB+ GPU
Memory saved by CPKD:                  18.1 GB  (44% reduction)
```

---

6. Why PyTorch Cannot Do This

Capability	PyTorch Distillation	CPKD
Teacher logits	Full [B,S,V] tensor in HBM	Never materialized (fused KL-CE)
Loss computation	Separate CE + KL kernels	Single fused kernel
Teacher-student scheduling	Sequential (one forward at a time)	Overlapped (two CUDA streams, cost-model guided)
Logit truncation k	User-specified hyperparameter	Compiler-selected via spectral analysis
Feature matching layers	User-specified	WGGO ILP-optimized
Student architecture	Manual design	CEP search (compilation oracle)
Backward generation	PyTorch autograd (saves all teacher activations)	Source AD (teacher backward absent)
Memory for teacher backward	~8 GB (activations saved)	0 (dead gradient elimination)

7. Implementation Plan

Component	Location	LOC (est.)	Description
Fused KL-CE kernel	`nsl-codegen/src/cpkd_fused_loss.rs`	600	Tiled matmul+softmax+KL+CE in one kernel
Teacher scheduler	`nsl-codegen/src/cpkd_schedule.rs`	400	Two-stream teacher-student overlap
Spectral k-selector	`nsl-codegen/src/cpkd_spectral.rs`	200	SVD-based effective vocab rank
Feature matching codegen	`nsl-codegen/src/cpkd_feature.rs`	300	Projection + cosine loss generation
CEP student design	`nsl-codegen/src/cpkd_student.rs`	300	CEP search for student architecture
Distill block codegen	`nsl-codegen/src/distill.rs`	400	Parse and compile `distill()` block
WGGO integration	`nsl-codegen/src/wggo.rs`	150	Distillation variables in ILP

Total: ~2,350 LOC. Dependencies: fused_linear_ce.rs, source_ad.rs, cep_oracle.rs, cost_model.rs, wggo.rs

---

8. Conclusion

CPKD completes the final gap in NSL's compiler-native optimization suite. The full set:

Paper	Domain	Unique Compiler Insight
WRGA	PEFT	Dead gradient elimination
CSHA	Attention	Boundary fusion across attention sublayers
FASE	Gradient Accumulation	Accumulation + optimizer fused at compile time
PCA	Sequence Packing	Document masks as kernel control flow
CEP	Pruning + NAS	Compiler IS the evaluation oracle
CPDT	Distributed Training	Joint sharding + precision + placement
WGGO	Global Solver	Hierarchical ILP over Wengert graph
CFIE	**Inference**	**Persistent kernels, fused sampling, compiled speculation**
CPKD	**Distillation**	**Fused teacher-student loss, logit-free KL divergence**

The nine papers share one thesis: **ML optimization is a compilation problem.** Every decision that existing frameworks make at runtime — from adapter placement to KV cache layout to teacher logit management — NSL makes at compile time. The compiled binary IS the globally-optimal training, distillation, and inference plan.

References

1. Hinton, G., et al. "Distilling the Knowledge in a Neural Network." NeurIPS Workshop 2015.
2. Gu, Y., et al. "MiniLLM: Knowledge Distillation of Large Language Models." ICLR 2024.
3. Muralidharan, S., et al. "Compact Language Models via Pruning and Knowledge Distillation." NeurIPS 2024.
4. Agarwal, R., et al. "On-Policy Distillation of Language Models." ICLR 2024.
5. Hsu, P., et al. "Liger-Kernel: Efficient Triton Kernels for LLM Training." 2024.